I/O

- One JSON command per line
- One JSON output
- Commands to stage (plan) changes, tick to commit them and advance “time”

High Level Chronological Flow:

- main.rs takes in one input line and calls protocol.rs, also starts up the World Engine
- protocol.rs handles json deserializing and serializing, identifies the command, parses parameters, and does parameter chcks, creates the world in init
- engine.rs takes the commands and executes
- protocol.rs takes the output of the engine function and builds ok or error response
- main.rs serializes the output and prints the JSON

- state.rs is where all the world’s data lives, but engine OWNS that

Models

- VesselState: Enum w/ Idle, Transit, and Docked states. Each state also includes relevant info. A vessel can’t be in multiple states at once, thus an enum is the best choice
- WorldState: Represents the world (used for both planned and committed version of the world). Has tick, vessels, and what’s at each dock
- DockCargo: Represents cargo at a dock, where it is, destination, and how many
    - Don’t need to track the individual cargo pieces themselves b/c we just care about how many need to goo from one dock to another. Essentially representing grouping into location + destination and then counting how many
- Cargo: represents cargo aboard a vessel, where it’s going and how many
    - Again, don’t need to store specifically where each is. Only care about how many are going to one place
- Engine: The actual world itself, hosts committed and planned worlds, history, travel_times (from init), dock throughput, and dock operations
- Serialize/Deserialize structs: How serde converts between JSON and Rust structs

Planned vs Committed

- Ticks affect the planned world, and then saves that into committed
- Committed transits are final, once a vessel is in transit it can't be commanded to move
- Planned destinations can be modified, but can't be once it's been committed (ticked)

Cargo

- Load/Unload moves only one unit
- Swap loads one and unloads one
- Throughput is shared at a dock, resets per tick
- Unloading cargo at its destination completes delivery and removes it from tracked inventory, unloading elsewhere returns it to that dock's inventory

Testing

- Send commands through protocol::process
- Check edge cases for param inputs / not allowed tick behavior
- Allow CLI tests as well